

Lab 05: Procedures and Stack

Lab Objectives:

1. Understand what a **procedure (subroutine)** is in assembly.
2. Learn how to **call and return** from procedures using CALL and RET.
3. Understand the **stack** and how registers/parameters are saved and restored.
4. Implement **modular programming** concepts at the assembly level.

Introduction

In this lab, students will explore **procedures (subroutines)** and **stack operations** in assembly language. Procedures allow code to be organized into reusable blocks, reducing repetition and improving readability. The **CALL** and **RET** instructions manage execution flow between the main program and procedures, while the **stack** provides temporary storage for register values, return addresses, and parameters. Understanding these instructions is essential for implementing modular programming concepts at the assembly level and for handling complex programs efficiently.

Key Concepts to Cover:

- **CALL** → jump to a procedure and push return address on stack.
- **RET** → return to the instruction after the last CALL.
- **PUSH / POP** → save and restore register values.
- Passing parameters via **registers** or **stack**.
- Local variables (optional, using stack offsets).

CALL and RET

- **CALL** → Like calling a function in C++, e.g., `foo()`; The program jumps to the function's code and remembers where to return.
- **RET** → Like reaching the end of a C++ function or using `return`; The program goes back to the point just after the function call.

So in assembly, **CALL + RET = function call and return** in C++.

First Program

```
; Main Program
MOV AX, 5      ; Initialize AX = 5
CALL ADD_TWO   ; Call procedure to add 2
; After return, AX should be 7

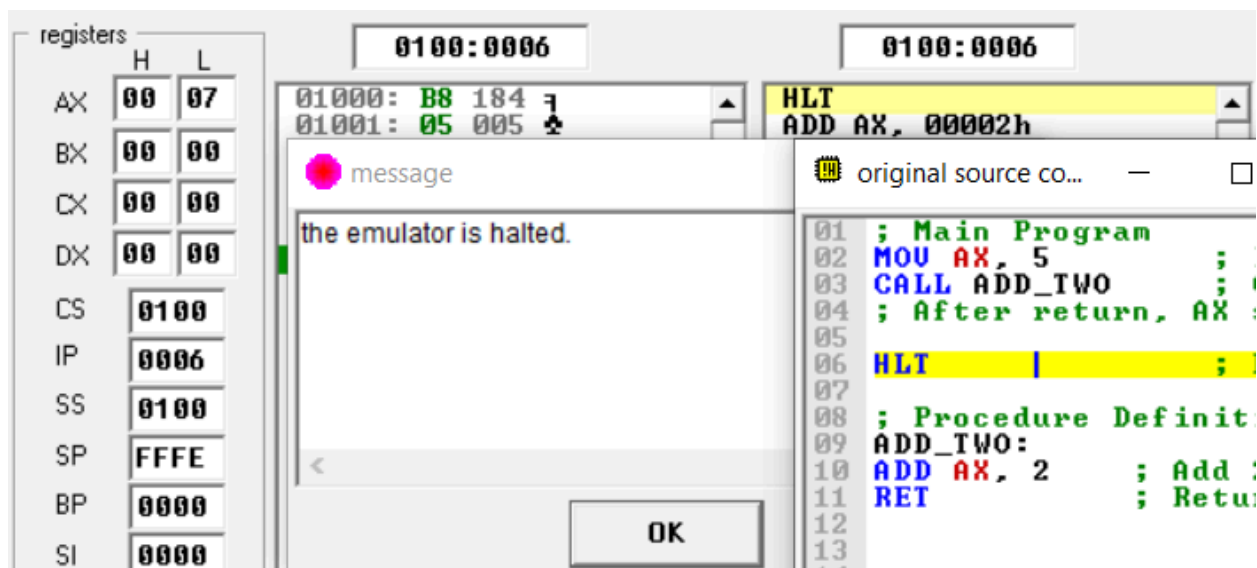
HLT           ; End program

; Procedure Definition
ADD_TWO:
    ADD AX, 2   ; Add 2 to AX
    RET        ; Return to main program
```

Explanation:

- **CALL ADD_TWO** → Like calling a function. CPU saves the return address on the stack.
- **Inside ADD_TWO:** → Procedure body executes, modifies AX.
- **RET** → Like returning from a function, CPU pops the saved address and resumes execution.

Output



Execution after return

```
MOV AX, 5
CALL ADD_TWO      ; CPU pushes address of "MOV BX, AX" and jumps to
ADD_TWO
MOV BX, AX        ; Execution resumes here after RET
HLT

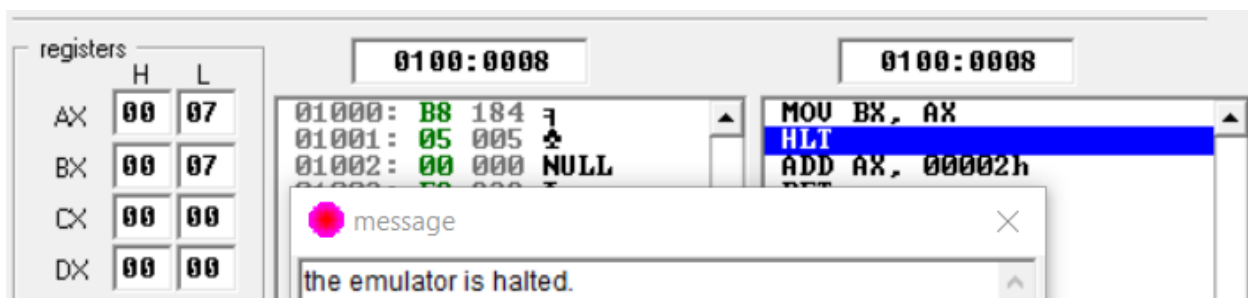
ADD_TWO:
    ADD AX, 2
    RET
```

Explanation:

When RET is executed, the CPU:

- Pops the return address from the stack.
- Resumes execution at the instruction located immediately after the original CALL.
- In this case, after RET, control returns to MOV BX, AX.

Output



Program without HLT

```
MOV AX, 5
CALL ADD_TWO      ; CPU pushes address of "MOV BX, AX" and jumps to
ADD_TWO
MOV BX, AX        ; Execution resumes here after RET

ADD_TWO:
    ADD AX, 2
    RET
```

Explanation:

If there is nothing after MOV BX, AX, then:

- The CPU will try to fetch the next instruction from memory.
- If the next memory location does not contain a valid instruction, the processor may:
 - execute garbage data as instructions (undefined behavior),
 - or eventually crash/halt depending on the environment.

Output

registers		0100:FF00		0100:FF00	
	H	L			
AX	00	09	10F00: F4 244	↑	HLT
BX	00	07	10F01: 00 000	NULL	ADD [BX + SI], AL
CX	00	00	10F02: 00 000	NULL	ADD [BX + SI], AL
DX	00	00	10F03: 00 000	NULL	ADD [BX + SI], AL
			10F04: 00 000	NULL	ADD [BX + SI], AL
			10F05: 00 000	NULL	ADD [BX + SI], AL
			10F06: 00 000	NULL	ADD [BX + SI], AL
			10F07: 00 000	NULL	ADD [BX + SI], AL

Multiple Procedures

```
MOV BX, 5      ; Initialize BX = 5
CALL ADD_TWO   ; Call first procedure ? BX = BX + 2
CALL MUL_THREE ; Call second procedure ? BX = BX * 3
; After both calls, BX = (5 + 2) * 3 = 21 or 15 in Hex

HLT            ; End program

ADD_TWO:
    ADD BX, 2  ; Add 2 to BX
    RET        ; Return to main program

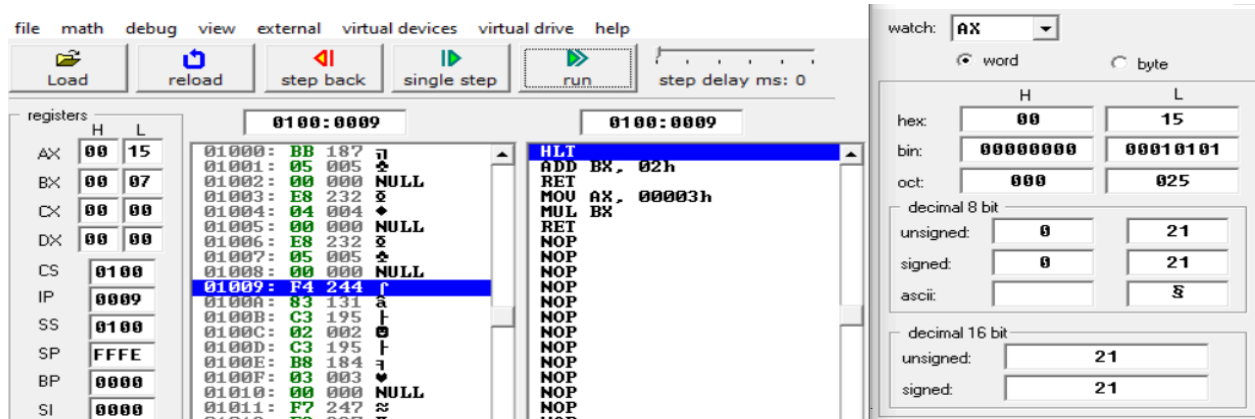
MUL_THREE:
    MOV AX, 3  ; MOV 3 to AX
    MUL BX     ; Multiply BX with AX which is 3
    RET        ; Return to main program
```

Explanation:

- The program shows how to use **multiple procedures** with CALL and RET.
- ADD_TWO adds 2 to AX, while MUL_THREE multiplies AX by 3.

- Control moves into each procedure with CALL and returns back after RET, like functions in C++.

Output



Difference Between JMP and CALL

- JMP:** Performs an unconditional jump to a label. Execution continues at the new location, but it does not remember where it came from (no return possible).
- CALL:** Jumps to a procedure and saves the return address (next instruction) on the stack. After RET, execution returns back to that saved point.

In C++ terms:

- JMP** is like a **goto** (no way back).
- CALL** is like a **function call** (you return after finishing).

Nested Procedure

```
MOV AX, 2
CALL PROC1
HLT
```

; First procedure

PROC1:

```
ADD AX, 3 ; AX = 2 + 3 = 5
```

```
CALL PROC2 ; Call another procedure
```

```
RET
```

; Second procedure

PROC2:

ADD AX, 4 ; AX = 5 + 4 = 9

RET

Output

registers		0100:0006		0100:0006	
	H L				
AX	00 09	01000: B8 184 7		HLT	
BX	00 00	01001: 02 002 0		ADD AX, 00003h	
CX	00 00	01002: 00 000 NULL		CALL 0000Eh	
DX	00 00	01003: E8 232 8		RET	
		01004: 01 001 0		ADD AX, 00004h	
		01005: 00 000 NULL		RET	
		01006: F4 244 ↑		NOP	
		01007: 05 005 *		NOP	

Pointer Registers:

Pointer registers are special registers in x86 assembly used to hold memory addresses. They act as *pointers* to specific locations in memory and help the CPU manage data movement, stack operations, and instruction execution.

- **SP (Stack Pointer):** Points to the top of the stack (last pushed value).
- **BP (Base Pointer):** Provides a stable reference inside procedures to access parameters/local variables on the stack.
- **IP (Instruction Pointer):** Holds the address of the next instruction to be executed.

STACK: In assembly, the stack is a special region of memory used to temporarily store data such as:

- Return addresses (saved automatically during a CALL).
- Procedure parameters (when you PUSH them manually).
- Local variables or temporary values.
- Register values you want to save and restore.

It works like a pile of plates:

- **PUSH** → put something on top of the stack.
- **POP** → take the top item back.

Simple Example

MOV AX, 5 ; AX = 5

PUSH AX ; push AX onto the stack

```
POP BX    ; pop top of stack into BX
HLT
```

Output

registers		0100:0005		0100:0005	
	H L				
AX	00 05	01000: B8 184	3	MOV AX, 00005h	
BX	00 05	01001: 05 005	5	PUSH AX	
CX	00 00	01002: 00 000	NULL	POP BX	
DX	00 00	01003: 50 080	P	HLT	
		01004: 5B 091	L	NOP	
		01005: F4 244	↑	NOP	
		01006: 90 144	E	NOP	

Pushing Multiple Values (LIFO)

```
MOV AX, 5    ; AX = 5
PUSH AX      ; push 5

MOV AX, 7    ; AX = 7
PUSH AX      ; push 7

POP BX       ; pop top of stack into BX
POP CX       ; pop next value into CX
HLT
```

Output

registers		0100:000A		0100:000A	
	H L				
AX	00 07	01000: B8 184	3	MOV AX, 00005h	
BX	00 07	01001: 05 005	5	PUSH AX	
CX	00 05	01002: 00 000	NULL	MOV AX, 00007h	
DX	00 00	01003: 50 080	P	PUSH AX	
		01004: B8 184	3	POP BX	
		01005: 07 007	BEEP	POP CX	
		01006: 00 000	NULL	HLT	

Using SP with a Procedure

```
MOV AX, 5
PUSH AX    ; push 5
CALL ADD_TWO ; pushes return address + jumps

ADD_TWO:
```

```

POP BX    ; pops return address (not 5!)
ADD BX, 2
MOV AX, BX
RET

```

Output

registers		0100:0005		0100:0005	
	H	L			
AX	00	09	01000: B8 184 7	ADD [BX + SI], AL	
BX	00	09	01001: 05 005 ♠	POP BX	
CX	00	00	01002: 00 000 NULL	ADD BX, 02h	
DX	00	00	01003: 50 080 P	MOV AX, BX	
			01004: E8 232 0	RET	
			01005: 00 000 NULL	NOP	
			01006: 00 000 NULL	NOP	

Using BP for safer access

```

MOV AX, 5
PUSH AX    ; push argument
CALL MYPROC
HLT

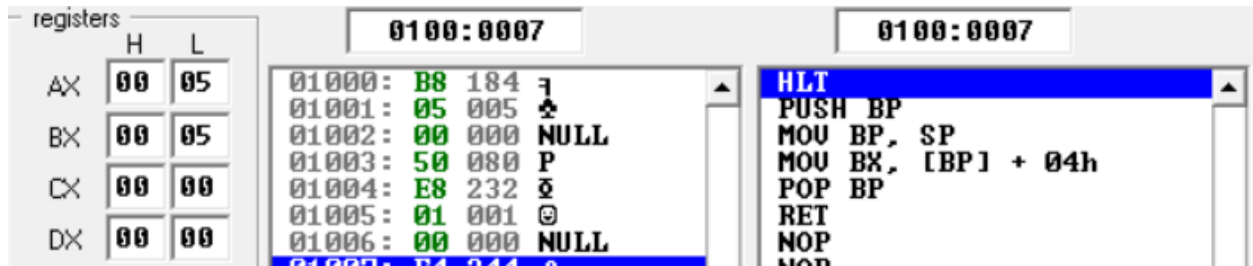
; Procedure
MYPROC:
    PUSH BP    ; save old BP
    MOV BP, SP    ; set BP as frame pointer
    MOV BX, [BP+4] ; get argument (5)
    POP BP      ; restore BP
    RET

```

Explanation:

- PUSH BP saves the old BP value on the stack.
- MOV BP, SP makes BP point to the current stack frame.
- [BP] = old BP, [BP+2] = return address, [BP+4] = first argument.
- Using BP keeps argument access stable even if SP changes.
- POP BP restores the old BP before returning.
- RET uses the saved return address to go back.

Output



Exercise:

Task 1: Push and Pop Once

- Put AX = 5.
- Push it on the stack.
- Pop into BX.
- Result: BX = 5.

Task 2: Two Values

- Put AX = 3 and BX = 7.
- Push both onto the stack.
- Pop into CX and DX.
- Result: CX = 7, DX = 3.

Task 3: CALL and RET Basic

- AX = 4 in main program.
- Call a procedure that adds 2.
- After RET, AX = 6.

Task 4: Save Register with PUSH/POP

- BX = 9 in main program.
- Push BX, change BX = 0.
- Pop BX back.
- Result: BX = 9.

Task 5: Swap Two Values Using Stack

- Push AX = 8 and BX = 12 onto the stack.
- Pop them back in reverse order.
- Result: AX = 12, BX = 8.